LESSON PLAN · GENERATIVE AI (GR 6–8) INTERMEDIATE: GEN-AI CREATORS · LEVEL 2

# Lesson 3: Planning a Program with AI

Module 1 | 60 minutes | Idea → pseudocode → code | Grades 7–11 · ages 12–16

**Objective:** Turn a game idea into pseudocode steps, ask an AI to review your plan, then build and test the quiz game yourself.

**Success criteria**

- I can turn an idea into pseudocode steps.
- I can ask an AI to review my plan without writing my code.
- I can find a missing step by tracing a plan.
- I can build and test a quiz game one step at a time.

## Key vocabulary

- **pseudocode**: A program written as plain English steps.
- **input**: Information the player types in; input() in Python, ask and wait in Scratch.
- **if / else**: Do one thing if something is true, otherwise do another.
- **score variable**: A variable that counts points, starting at 0.
- **edge case**: An unusual input that might break the program, like capitals or an empty answer.
- **trace**: Walk through a plan or program step by step to see what happens.

## You will need

- Projector or screen share with Kitabash and the AI tools for this lesson
- Devices: 1 per learner (laptop, tablet or phone)
- Learner accounts only where allowed: see the course safety notes
- Printed worksheet (optional)
- Project files: l03-quiz-game-kit.md, l03-quiz-game.py, l03-teacher-demo-script.md

## Timed lesson plan (60 min)

| Time | Stage | What happens |
|---|---|---|
| 5 min | **Warm-up game** | Robot waiter: the class writes steps for a “robot” learner, who follows them literally until the plan is fixed. |
| 4 min | **Watch** | Play the lesson video; pause on the plan with a hole and ask everyone to find it. |
| 3 min | **Thumbs check** | Thumbs up / down: “The AI should write my code for me.” (Down — AI reviews the plan, you build it.) |
| 7 min | **Live demo** | Teacher demo: idea, AI pseudocode plan, trace and catch the hole, build question 1 and run it. |

| | | |
|---|---|---|
| 6 min | **Prompt Lab + plan (pairs)** | Pairs write their own pseudocode, then the review-my-plan prompt; check and improve it. |
| 4 min | **Quick competition** | Order the plan and the “you or AI?” sort — first three all-correct win. |
| 18 min | **Build** | Everyone builds the quiz game in the Python Lab (or Scratch), running after each question and doing the four player tests. Teacher circulates. |
| 6 min | **Show-and-tell** | Volunteers share their screen and the class plays their quiz. Everyone tries to break it with capitals or an empty answer. |
| 7 min | **Probe + exit ticket** | New situation: the capital-letter bug, then the exit ticket. |

## Unplugged warm-up

Robot waiter (5 min): one learner is a “robot” who only follows exact steps. The class writes pseudocode on the board to make the robot serve a glass of water. The robot follows it literally — and fails at every missing step (“pick up the glass” before “walk to the table”?). Fix the steps together.

## Teaching the idea

Planning is the most underrated programming skill. Learners write the idea in one sentence, break it into pseudocode (plain English steps), then ask AI to REVIEW the plan — missing steps, tricky cases — but not to write the code. AI plans can have holes (a score that never goes up, a final score shown too early), so learners trace the plan with a pretend player. Then they build one step at a time, running after each step, and test like a player (right, wrong, capitals, empty). Python Lab learners use the checker; Scratch learners use ask and wait, if/else and a score variable.

**Analogy:** Pseudocode is like a recipe written before you cook: the AI can read it and say “you forgot the salt”, but you still do the cooking.

**Live demo:** Shared screen: Kitabash AI helper + Python Lab (or Scratch). 1) Write the idea: “a 3-question India quiz that keeps score.” 2) Type: “Give me a plan in pseudocode only, in numbered steps. Do not write any Python code.” 3) Trace the plan with a pretend player on the board. If it forgets to add to the score (or shows the score too early), say “Caught it!” and fix the step. If the plan is complete, ask “What would happen with DELHI in capitals?” and add a step. 4) Build question 1 only, run it with “Delhi” and “Mumbai”. 5) Show copy-and-change for question 2, then run the four player tests.

## Questions to ask

| Ask | Listen for |
|---|---|
| What is pseudocode? | Your program written as plain English steps. |
| Why ask AI to review your plan instead of writing code? | You learn more, keep ownership, and AI is good at spotting gaps. |

| | |
|---|---|
| Where must “add 1 to score” go? | Inside the part that runs when the answer is right. |
| Name two tests for a quiz game. | All right, all wrong, capital letters, empty answer. |
| Why run the code after each question? | So a bug is found right away, in the small part you just added. |

## Differentiation

**Support**
Give a pseudocode frame with blanks (“Set ___ to 0”, “Ask ___”, “If ___ then ___”). Learners can build only one or two questions and still pass the plan review.

**Stretch**
Add a fourth question with two possible right answers, a “Perfect!” message for 3 out of 3, or store the questions in a list and loop over them. Ask the AI to review the new plan first.

## Common misconceptions

| Misconception | How to fix it |
|---|---|
| The AI plan is always complete | AI plans can miss steps. Trace them with a pretend player. |
| Pseudocode has to look like real code | It is plain English. Anyone should be able to read it. |
| I should write all the code, then run it once at the end | Run after each small step so bugs are easy to find. |
| If input() asks for a number, I can compare with 11 (no quotes) | input() always gives text. Compare with "11", or convert with int(). |

## Assessment

Use the Prompt Lab results (auto-checked), the quick checks and the Probe answers. Written answers arrive in your review queue.

**Exit ticket**
Name one step an AI plan for a quiz game might forget, and how you would catch it.

**Homework**
Play your quiz with someone at home. Add one question about something they like, and record one test that failed and how you fixed it.

## Answer key — Kitabash activities

### 1. Prompt Lab: review my plan (perform · ai lab)

**Checklist:** Describe the game · Share your own plan · Ask for missing steps · Ask about tricky cases · Say no code

**Strong example:**

```
I am making a 3-question quiz game about cricket that keeps score. Here is my plan in pseudocode: [my steps]. Please review my steps and tell me if anything is missing, and what tricky cases I should test, like capital letters or an empty answer. Do not write any code — I will build it myself.
```

### 2. Order the plan (perform · ordering)

**Order:** Set score to 0 → Ask the question and store the answer → If the answer is right, say Correct and add 1 to score → Otherwise, show the right answer → After all questions, show the final score

### 3. Python Lab: my quiz game (perform · python lab)

**Goals checked:** All right answers score 3 · All wrong answers score 0 · Capital letters still count · Uses input, if and else

**One solution:**

```
# India quiz - planned with AI help, coded by me
score = 0
print("Welcome to the India Quiz!")

answer = input("What is the capital of India? ")
if answer.lower() == "new delhi" or answer.lower() == "delhi":
    print("Correct!")
    score = score + 1
else:
    print("Not quite. It is New Delhi.")

answer = input("How many players does a cricket team have on the field? ")
if answer == "11":
    print("Correct!")
    score = score + 1
else:
    print("Not quite. It is 11.")

answer = input("Which festival is called the festival of lights? ")
if answer.lower() == "diwali":
    print("Correct!")
    score = score + 1
else:
    print("Not quite. It is Diwali.")

print("Your score: " + str(score) + " out of 3")
```

**4. Who should do it: you or the AI?** (perform · categorize)

**You do it:** Choose the game idea, Type the Python code, Run the tests and record results
**AI can help:** Spot missing steps in your plan, Suggest tricky test cases, Explain an error message

**5. New situation: the capital-letter bug** (probe · multiple choice)

**Answer:** Write a bug report: expected Correct for “Diwali”, got Not quite — then try .lower()

## Answer key — worksheet

1. **Key words:** pseudocode = A program written as plain English steps. · input = Information the player types in; input() in Python, ask and wait in Scratch. · if / else = Do one thing if something is true, otherwise do another. · score variable = A variable that counts points, starting at 0.
2. **Prompt Lab: review my plan:** Include: Describe the game; Share your own plan; Ask for missing steps; Ask about tricky cases; Say no code. Example: I am making a 3-question quiz game about cricket that keeps score. Here is my plan in pseudocode: [my steps]. Please review my steps and tell me if anything is missing, and what tricky cases I should test, like capital letters or an empty answer. Do not write any code — I will build it myself.
3. **Order the plan:** 1 Set score to 0 · 2 Ask the question and store the answer · 3 If the answer is right, say Correct and add 1 to score · 4 Otherwise, show the right answer · 5 After all questions, show the final score
4. **Plan your program:** Goals: All right answers score 3; All wrong answers score 0; Capital letters still count; Uses input, if and else. One solution: # India quiz - planned with AI help, coded by me score = 0 print("Welcome to the India Quiz!") answer = input("What is the capital of India? ") if answer.lower() == "new delhi" or answer.lower() == "delhi": print("Correct!") score = score + 1 else: print("Not quite. It is New Delhi.") answer = input("How many players does a cricket team have on the field? ") if answer == "11": print("Correct!") score = score + 1 else: print("Not quite. It is 11.") answer = input("Which festival is called the festival of lights? ") if answer.lower() == "diwali": print("Correct!") score = score + 1 else: print("Not quite. It is Diwali.") print("Your score: " + str(score) + " out of 3")
5. **Who should do it: you or the AI?:** You do it: Choose the game idea, Type the Python code, Run the tests and record results · AI can help: Spot missing steps in your plan, Suggest tricky test cases, Explain an error message

Kitabash original curriculum. Product names (ChatGPT, Gemini, Claude, Copilot, Canva, NotebookLM, Perplexity, Teachable Machine) belong to their owners; Kitabash is not affiliated with or endorsed by them. Tools and age rules change: check each tool’s current terms before class.